Lesson notes · for you

Shortest paths

Optimization · Level 3 · Lesson 2

Time
20 min

Materials
Paper and a pen; A rough map of roads near your home or school; Optional: a phone with Darsi to run your program

Learning goals

- Store a road map as a Python dictionary.
- Follow and complete Dijkstra's method.
- Explain why settling the closest place first is safe.

Activity: My map, settled

1. Draw (5 min): Draw six places near your school as dots, join the ones with a road, and write a rough distance on each road.
2. Settle (8 min): Write 0 by your home and a big number by every other place. Repeatedly circle the smallest uncircled number and update its neighbours, until the school is circled.
3. Code (5 min): Type your map as a dictionary into your program from this lesson and run it. Does it print the same distance to school?
4. Close a road (2 min): Cross out one road and run it again. Which place's distance grows the most?

Use made-up names for homes. Distances can be minutes of walking instead of kilometres.

Discussion

- Why is the straightest-looking road not always the shortest?
- What would go wrong with a negative road length?
- Who should decide which road gets repaired first?

Check yourself

- The one line that relaxes a road + why the smallest goes first.

Try it this week
Draw a map of the roads between your home, two friends' homes and school, with rough distances in minutes or kilometres. Run Dijkstra's method on paper, writing each place's number as it changes, then put the map into your program and check that it agrees.

Worksheet

Shortest paths

Optimization · Level 3 · Lesson 2

Name _____ Class _____ Date _____

1 **A friend's program settles places in list order (home, Tala, Kesi, school) instead of the smallest number first. It says school is 7 km away; your Dijkstra says 6. Why?**

- ◯ It settled Tala at 4 before the shorter way via Kesi was found
- ◯ One of the roads is negative
- ◯ Python added the numbers wrongly

2 **Put Dijkstra's method in order.**

- ☐ Relax its roads: lower any neighbour's number you can
- ☐ Repeat until the school is settled
- ☐ Give home 0 and every other place a very large number
- ☐ Settle it: its number is final
- ☐ Pick the waiting place with the smallest number

3 **Dijkstra's method can give a wrong answer if some roads have negative lengths.**

- ◯ True
- ◯ False

4 **Complete Dijkstra's method: after taking the waiting place p with the smallest number, relax each of its roads, so that dist[q] keeps the smaller of its old value and dist[p] + km. The program prints the shortest distance to school.**

```
roads = {"home": {"tala": 4, "kesi": 1},
         "kesi": {"tala": 2, "school": 7},
         "tala": {"school": 3},
         "school": {}}
dist = {p: 99 for p in roads}
dist["home"] = 0
todo = list(roads)
while todo:
    p = min(todo, key=lambda q: dist[q])
    todo.remove(p)
    for q, km in roads[p].items():
        pass  # keep the smaller distance
print(dist["school"])
```

Write your program:

5 **This program says the school is 7 km away, but the shortest way is 6. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
roads = {"home": {"tala": 4, "kesi": 1},
         "tala": {"school": 3},
         "kesi": {"tala": 2, "school": 7},
         "school": {}}
dist = {p: 99 for p in roads}
dist["home"] = 0
todo = ["home", "tala", "kesi", "school"]
while todo:
    p = todo[0]
    todo.remove(p)
    for q, km in roads[p].items():
        dist[q] = min(dist[q], dist[p] + km)
print(dist["school"])
```

✎ **Which road or path near you would you fix first, and who would gain most? Would the shortest paths agree with you?**

Check your answers

Shortest paths

1 **It settled Tala at 4 before the shorter way via Kesi was found**

In list order, Tala is settled at 4 and gives school 4 + 3 = 7.

2 **Top to bottom: 4, 5, 1, 3, 2**

Start with what you know (home is 0), and each turn make the smallest number final before using its roads.

3 **True**

Settling the smallest number is safe only because every other way is at least as long.

4

```
roads = {"home": {"tala": 4, "kesi": 1},
         "kesi": {"tala": 2, "school": 7},
         "tala": {"school": 3},
         "school": {}}
dist = {p: 99 for p in roads}
dist["home"] = 0
todo = list(roads)
while todo:
    p = min(todo, key=lambda q: dist[q])
    todo.remove(p)
    for q, km in roads[p].items():
        dist[q] = min(dist[q], dist[p] + km)
print(dist["school"])
```

Kesi is settled first (1) and lowers Tala to 3 and the school to 8; then Tala lowers the school to 6.

5 **Line 9: `p = min(todo, key=lambda q: dist[q])`**

todo[0] settles places in list order, so Tala (4) is settled before Kesi (1) can lower it to 3, and the school is stuck at 4 + 3 = 7.